Oliver Krauss

Modern Code - Arrays Basics

ModernCode | MTD.ba VZ SS25

Hagenberg 10.11.2025

HAGENBERG | LINZ | STEYR | WELS

FH OBERÖSTERREICH

UNIVERSITY OF APPLIED SCIENCES UPPER AUSTRIA

Motivation: Arrays

- Arrays enable organizing many values under a single name.
- They support efficient, systematic processing with loops.

Why Arrays Matter: Real Examples

- **Test Scores**: Instead of 30 separate variables, one array holds all scores
- **Game Levels**: Store difficulty settings for 50 levels in one place
- **Sensor Data**: Collect temperature readings every hour for a week
- **Inventory**: Track quantities of 100 different products

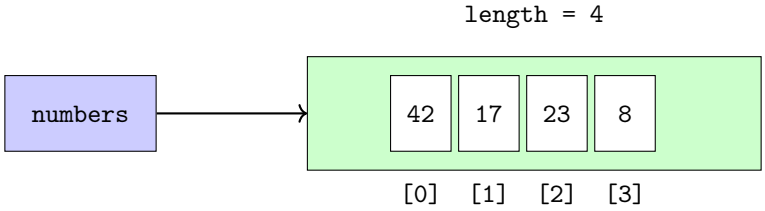
The Problem Arrays Solve

- **Without arrays**: 100 variables for 100 test scores
  - `int score1, score2, score3, ... score100;`
  - `int total = score1 + score2 + score3 + ... + score100;`
- **With arrays**: One variable, one loop
  - `int[] scores = new int[100];`
  - `for (int i = 0; i < 100; i++) total += scores[i];`

1D Arrays Overview

- Fixed-size, homogeneous collection of elements.
- Index-based access with a clear bounds mindset (0 to `length-1`).
- Concept: variable holds a reference to the array object.

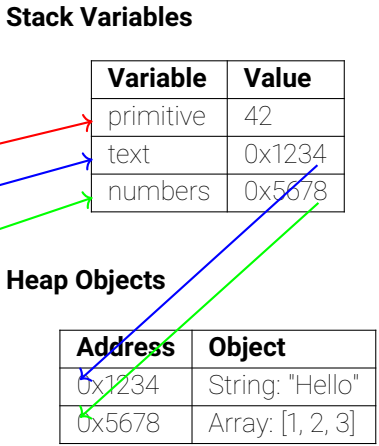
Visualizing Arrays: What They Look Like



Key Point: The variable `numbers` points to the array object, which contains the actual values.

Reminder: Arrays live on the heap

```
1   static void example() {
2   int primitive = 42;
3   String text = "Hello";
4   int[] numbers = {1, 2, 3};
5   }
```



Syntax: Declaring and Initializing

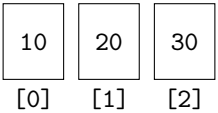
```
1  // Declaration and allocation
2  int[] scores = new int[5]; // default values are 0
3
4  // Declaration with initializer
5  int[] levels = {1, 2, 3, 4};
6
7  // Separate declaration and later allocation
8  double[] temperatures;
9  temperatures = new double[7];
```

Understanding Array Declaration

- `int[] scores` - Declares a variable that can hold an array of integers
- `new int[5]` - Creates a new array object with space for 5 integers
- `=` - Assigns the array object to the variable
- **Important**: The variable holds a *reference*, not the values themselves

Indexing: The Key Concept

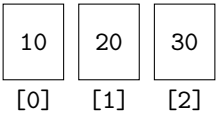
- **Indices start at 0**, not 1!
- For an array of length `n`, valid indices are `0` to `n-1`
- `data[0]` = first element, `data[1]` = second element, etc.
- **Common mistake**: Trying to access `data[data.length]` (this will crash!)



Array Length

- `array.length` gives you the **number of elements** in the array
- `length` is a **property**, not a method (no parentheses!)
- For array `data = {10, 20, 30}`:
  - `data.length` equals `3`
  - The array has 3 elements
  - Valid indices are `0`, `1`, and `2`
- **Key insight**: `length` tells you how many elements exist, not the highest valid index

```
Valid indices:  0, 1, 2
      length = 3
```



Length and Indexing in Action

```
1  int[] data = {10, 20, 30};
2  int n = data.length;   // 3
3
4  // Read element at index 1
5  int second = data[1];  // 20
6
7  // Write/update element at index 2
8  data[2] = 99;          // data is now {10, 20, 99}
```

Iteration Patterns

- Index-based **`for`** loop for full control over indices.
- Enhanced **`for`** (for-each) for read-only traversal.
- Reverse traversal and stepping by a fixed stride.

Pattern: Index-Based For Loop

```
int[] data = {10, 20, 30, 40, 50};

// Full control over indices
for (int i = 0; i < data.length; i++) {
    IO.println("Index " + i + ": " + data[i]);
    // Can also modify: data[i] = data[i] * 2;
}
```

When to use:

- Need to know the index position
- Want to modify array elements
- Need to access multiple arrays with same index
- Want to control the iteration order

Pattern: Enhanced For Loop (For-Each)

```
1   int[] data = {10, 20, 30, 40, 50};
2
3   // Read-only traversal - no index needed
4   for (int value : data) {
5       IO.println("Value: " + value);
6       // Cannot modify: value = 0; // This won't work!
7   }
8
```

When to use:

- Only need the values, not the indices
- Read-only operations (sum, count, find, etc.)
- Cleaner, more readable code
- Less chance of off-by-one errors

Pattern: Reverse Traversal

```
int[] data = {10, 20, 30, 40, 50};

// Process from end to beginning
for (int i = data.length - 1; i >= 0; i--) {
    IO.println("Reverse: " + data[i]);
}

```

When to use:

- Need to process elements from last to first
- Building something in reverse order
- Checking conditions from the end
- When the last elements are more important

Pattern: Stride Traversal

```
1   int[] data = {10, 20, 30, 40, 50, 60, 70, 80};
2
3   // Process every 2nd element
4   for (int i = 0; i < data.length; i += 2) {
5       IO.println("Every 2nd: " + data[i]);
6   }
```

When to use:

- Want to skip elements (every 2nd, 3rd, etc.)
- Processing alternate elements
- Working with data that has a pattern
- Reducing the number of iterations

New Concept: The Ternary Operator

```
1   // Traditional if-else
2   int max;
3   if (a > b) {
4       max = a;
5   } else {
6       max = b;
7   }
8
9   // Ternary operator (
        shorthand)
10  int max = (a > b) ? a : b;
```

Syntax: `condition ? valueIfTrue : valueIfFalse`

- `condition`: Boolean expression to evaluate
- `?`: Separates condition from values
- `valueIfTrue`: Result if condition is true
- `:`: Separates true and false values
- `valueIfFalse`: Result if condition is false

Example: Sum and Average

```
1   int[] arr = {5, 7, 9};
2   int sum = 0;
3   for (int i = 0; i < arr.length; i++) {
4       sum += arr[i];
5   }
6
7   double average;
8   average = (arr.length == 0) ? 0.0 : (double) sum / arr.length;
```

Note: Cast to `double` to avoid integer division.

Understanding the Sum Pattern

- **Initialize accumulator**: `sum = 0`
- **Loop through array**: Add each element to the accumulator
- **Why start with 0?** Adding 0 doesn't change the sum
- **Why cast to double?** `5/2` gives 2, but `(double)5/2` gives 2.5

Processing: Min and Max

```
1   int[] a = {4, -2, 7, 1};
2   int min = a[0];
3   int max = a[0];
4   for (int i = 1; i < a.length; i++) {
5       if (a[i] < min) min = a[i];
6       if (a[i] > max) max = a[i];
7   }
8
```

Understanding the Min/Max Pattern

- **Initialize with first element**: `min = a[0]` (not 0!)
- **Why not start with 0?** What if all numbers are negative?
- **Start loop at 1**: We already checked the first element
- **Update when better found**: Only change if current element is better

Built-in Alternatives: Math Methods

```
1   int[] arr = {4, -2, 7, 1};
2
3   // Built-in Math methods (for 2 values)
4   int minOfTwo = Math.min(arr[0], arr[1]);
5   int maxOfTwo = Math.max(arr[0], arr[1]);
6
7   // Note: Math.min/max only work with 2 values at a time
8
```

Processing: Counting and Accumulation

```
1  int[] readings = {12, 18, 21, 17, 25};
2  int threshold = 20;
3  int countAbove = 0;
4  for (int value : readings) {
5      if (value > threshold) {
6          countAbove++;
7      }
8  }
9
```

Understanding the Counting Pattern

- **Initialize counter**: `countAbove = 0`
- **Check condition**: `if (value > threshold)`
- **Increment when true**: `countAbove++`
- **Enhanced for loop**: Perfect when you don't need the index

Searching: Contains (Boolean Search)

```
1   int[] nums = {3, 8, 2, 8};
2   int target = 8;
3
4   // Check if target exists in array
5   boolean found = false;
6   for (int x : nums) {
7       if (x == target) {
8           found = true;
9           break;
10      }
11  }
12  IO.println("Found: " + found);
```

Searching: First Index (Position Search)

```
1   int[] nums = {3, 8, 2, 8};
2   int target = 8;
3   // Find first occurrence of target
4   int index = -1;
5   for (int i = 0; i < nums.length; i++) {
6       if (nums[i] == target) {
7           index = i;
8           break;
9       }
10  }
11  if (index != -1) {
12      IO.println("Found at index: " + index);
13  }
```

Understanding Search Patterns

- **Boolean search**: Just need to know if something exists
- **Index search**: Need to know where it is (or that it's not found)
- **Why -1 for not found?** -1 is never a valid array index
- **Why `break`?** Stop searching once we find it

Input & Validation

- Validate array sizes before allocation.
- Access only indices from 0 to `length-1`.
- Be careful with off-by-one errors in loops.

Safe Array Creation

```
1   // Get size from user
2   Scanner input = new Scanner(System.in);
3   IO.print("How many scores? ");
4   int size = input.nextInt();
5
6   // Validate size
7   if (size <= 0) {
8       IO.println("Size must be positive!");
9       return;
10  }
11  // Create array
12  int[] scores = new int[size];
```

Example: Test Scores Analysis

```
1   int[] scores = {85, 92, 78, 96, 88};
2   int sum = 0;
3   int max = scores[0];
4   int min = scores[0];
5   for (int score : scores) {
6       sum += score;
7       if (score > max) max = score;
8       if (score < min) min = score;
9   }
10  double average = (double) sum / scores.length;
11  IO.println("Average: " + average);
12  IO.println("Range: " + (max - min));
```

What This Example Shows

- **Single loop, multiple operations**: Calculate sum, min, and max together
- **Proper initialization**: Start min/max with first element
- **Type casting**: Ensure accurate average calculation
- **Enhanced for loop**: When index isn't needed

Common Mistakes Overview

What we'll cover: Four common pitfalls when working with arrays

- **Off-by-one errors** in loop bounds
- **Array length syntax** confusion
- **Integer division** surprises
- **Min/max initialization** problems

Why this matters: These mistakes cause runtime crashes, compilation errors, and incorrect results. Learning to avoid them will make your code more robust and reliable.

Off-by-One Errors

Problem: Using `<=` instead of `<` in loop bounds

Explanation: Arrays are zero-indexed, so valid indices are 0 to length-1. Using `<=` will try to access index `length`, causing an `ArrayIndexOutOfBoundsException`.

```
int[] data = {1, 2, 3, 4, 5};
// WRONG: This will crash!
for (int i = 0; i <= data.length; i++) {
    IO.println(data[i]);  // Crashes at i=5
}
// CORRECT: Use < instead of <=
for (int i = 0; i < data.length; i++) {
    IO.println(data[i]);  // Works for i=0,1,2,3,4
}
```

Array Length Syntax

Problem: Using `.length()` instead of `.length`
Explanation: `length` is a field (property) of arrays, not a method. Adding parentheses `()` makes Java think you're calling a method, which doesn't exist.

```
int[] data = {1, 2, 3, 4, 5};

// WRONG: Won't compile!
int size = data.length();  // Error: cannot find symbol

// CORRECT: Use .length without parentheses
int size = data.length;    // Works fine
```

Integer Division Surprises

Problem: Integer division truncates decimal results

Explanation: When dividing integers, Java truncates the decimal part. This can lead to unexpected results when calculating averages or percentages.

```
int[] scores = {85, 90, 78, 92, 88};
int sum = 0;
// Calculate sum
for (int i = 0; i < scores.length; i++) {
    sum += scores[i];
}
// WRONG: Integer division truncates
int average = sum / scores.length;  // 433 / 5 = 86 (truncated)
// CORRECT: Use double for accurate division
double correct = (double) sum / scores.length;  // 433.0 / 5 = 86.6
```

Min Initialization Problem

Problem: Initializing min with 0 when all numbers are positive

Explanation: If you start with `min = 0` and all numbers in your array are greater than 0, the min variable will never be updated, giving you the wrong result.

```
int[] numbers = {15, 8, 22, 3, 19};
// WRONG: min will never change
int min = 0;
// CORRECT: Start with first element
int min = numbers[0];  // min = 15
for (int i = 1; i < numbers.length; i++) {
    if (numbers[i] < min) min = numbers[i];
}
```

Max Initialization Problem

Problem: Initializing max with 0 when all numbers are negative

Explanation: If you start with `max = 0` and all numbers in your array are less than 0, the max variable will never be updated, giving you the wrong result.

```
int[] numbers = {-15, -8, -22, -3, -19};
// WRONG: max will never change
int max = 0;
// CORRECT: Start with first element
int max = numbers[0];  // max = -15
for (int i = 1; i < numbers.length; i++) {
    if (numbers[i] > max) max = numbers[i];
}
```

Summary: Arrays Basics

Array Fundamentals

- Declaration, allocation, initialization
- Zero-based indexing (0 to length-1)
- Variables hold references to heap objects

Iteration Patterns

- Index-based (full control, can modify)
- Enhanced for (read-only, cleaner code)
- Reverse and stride traversal

Core Operations

- Sum/average with type casting
- Min/max with proper initialization
- Counting and accumulation

Searching

- Boolean (contains) vs Index (position)
- Early termination with break
- Return -1 for not found

Common Mistakes

- Off-by-one errors (<= vs <)
- .length() vs .length
- Integer division surprises
- Wrong min/max initialization

Key Insights

- Arrays enable systematic processing
- Choose the right loop pattern
- Always validate bounds